

Здесь будет титульник, листай ниже

СОДЕРЖАНИЕ

1 ПОСТАНОВКА ЗАДАЧИ.....	6
1.1 Описание входных данных.....	8
1.2 Описание выходных данных.....	8
2 МЕТОД РЕШЕНИЯ.....	10
3 ОПИСАНИЕ АЛГОРИТМОВ.....	13
3.1 Алгоритм конструктора класса Employee.....	13
3.2 Алгоритм метода getName класса Employee.....	13
3.3 Алгоритм метода showRole класса Employee.....	13
3.4 Алгоритм конструктора класса Developer.....	14
3.5 Алгоритм метода showTasks класса Developer.....	14
3.6 Алгоритм метода showRole класса Developer.....	15
3.7 Алгоритм конструктора класса Manager.....	15
3.8 Алгоритм метода showProjects класса Manager.....	15
3.9 Алгоритм метода showRole класса Manager.....	16
3.10 Алгоритм конструктора класса TeamLead.....	16
3.11 Алгоритм метода showLevel класса TeamLead.....	16
3.12 Алгоритм метода showLeadName класса TeamLead.....	17
3.13 Алгоритм метода setName класса TeamLead.....	17
3.14 Алгоритм метода showRole класса TeamLead.....	18
3.15 Алгоритм функции main.....	18
4 БЛОК-СХЕМЫ АЛГОРИТМОВ.....	20
5 КОД ПРОГРАММЫ.....	25
5.1 Файл Developer.cpp.....	25
5.2 Файл Developer.h.....	25
5.3 Файл Employee.cpp.....	26
5.4 Файл Employee.h.....	26

5.5 Файл main.cpp.....	27
5.6 Файл Manager.cpp.....	28
5.7 Файл Manager.h.....	28
5.8 Файл TeamLead.cpp.....	29
5.9 Файл TeamLead.h.....	30
6 ТЕСТИРОВАНИЕ.....	31
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ.....	32

1 ПОСТАНОВКА ЗАДАЧИ

В информационной системе технологической компании рассматриваются сотрудники разных ролей. Базовым объектом является сотрудник. На его основе определяются две роли: разработчик и менеджер. Также существует роль тимлида, которая совмещает функции разработчика и менеджера. Для моделирования такой системы необходимо построить ромбовидную иерархию классов.

Базовый класс "Работник"

В защищённом разделе:

- строковое свойство имени сотрудника.

В открытом разделе:

- конструктор с одним строковым параметром;
- метод вывода имени сотрудника в формате "Employee::showName -> <имя>";
- виртуальный метод вывода роли в формате "Employee::showRole -> Employee".

Промежуточный класс "Разработчик"

Класс должен:

- наследоваться от класса Работник так, чтобы избежать ошибки;
- содержать закрытое целочисленное свойство количества выполненных задач;
- иметь конструктор;
- иметь метод вывода количества задач в формате "Developer::showTasks -> <значение>";
- переопределять виртуальный метод вывода роли в формате "Developer::showRole -> Developer".

Промежуточный класс "Менеджер"

Класс должен:

- наследоваться от класса Работник так, чтобы избежать ошибки;
- содержать закрытое целочисленное свойство количества проектов;
- иметь конструктор;
- иметь метод вывода количества проектов в формате

"Manager::showProjects -> <значение>";

- переопределять виртуальный метод вывода роли в формате "Manager::showRole -> Manager".

Производный класс "Тимлид"

Класс должен:

- наследоваться от классов Разработчик и Менеджер;
- содержать закрытое целочисленное свойство уровня руководства;
- иметь конструктор;
- иметь метод вывода уровня руководства в формате

"TeamLead::showLevel -> <значение>";

- иметь метод вывода имени сотрудника из класса Тимлид в формате "TeamLead::showLeadName -> <имя>";

- иметь метод изменения имени сотрудника;
- переопределять виртуальный метод вывода роли в формате

"TeamLead::showRole -> TeamLead".

Алгоритм работы программы

1. Ввести имя сотрудника, количество выполненных задач, количество проектов, уровень руководства.

2. Ввести новое имя.

3. Создать объект производного класса, представляющего тимлида, используя введенные данные.

4. Вывести имя сотрудника через интерфейс базового класса.

5. Вывести имя сотрудника через интерфейс производного класса.
6. Изменить имя сотрудника с использованием функциональности производного класса.
7. Повторно вывести имя сотрудника через интерфейс базового класса.
8. Повторно вывести имя сотрудника через интерфейс производного класса.
9. Вывести количество выполненных задач.
10. Вывести количество проектов.
11. Вывести уровень руководства.
12. Создать три указателя:
 - 12.1. на объект, рассматриваемый как разработчик;
 - 12.2. на объект, рассматриваемый как менеджер;
 - 12.3. на объект, рассматриваемый как сотрудник.
13. Направить все указатели(присвоить адрес) на созданный объект тимлида.
14. Через каждый из указателей выполнить вызов метода вывода роли сотрудника.

1.1 Описание входных данных

Ivan 12 3 2
Petr

1.2 Описание выходных данных

```
Employee::showName -> Ivan  
TeamLead::showLeadName -> Ivan  
Employee::showName -> Petr  
TeamLead::showLeadName -> Petr  
Developer::showTasks -> 12  
Manager::showProjects -> 3  
TeamLead::showLevel -> 2
```

```
TeamLead::showRole -> TeamLead  
TeamLead::showRole -> TeamLead  
TeamLead::showRole -> TeamLead
```

2 МЕТОД РЕШЕНИЯ

Для решения задачи используется:

- объект `tl` класса `TeamLead` предназначен для описания объекта класса;
- объект `cin` класса `istream` предназначен для ввода значения с клавиатуры.

Класс `Employee`:

- свойства/поля:
 - поле имя сотрудника:
 - наименование — `name`;
 - тип — `string`;
 - модификатор доступа — `protected`;
- функционал:
 - метод `Employee` — конструктор с параметром;
 - метод `getName` — метод для получения значения защищённого свойства;
 - метод `showRole` — виртуальный метод для вывода роли.

Класс `Developer`:

- свойства/поля:
 - поле количество выполненных задач:
 - наименование — `ntasks`;
 - тип — `int`;
 - модификатор доступа — `private`;
- функционал:
 - метод `Developer` — конструктор с параметрами;
 - метод `showTasks` — метод для вывода закрытого свойства;

- о метод showRole — метод для вывода на экран роли сотрудника.

Класс Manager:

- свойства/поля:
 - о поле количество проектов:
 - наименование — nprojects;
 - тип — int;
 - модификатор доступа — private;
- функционал:
 - о метод Manager — конструктор с параметрами;
 - о метод showProjects — метод для вывода закрытого свойства;
 - о метод showRole — метод для вывода роли.

Класс TeamLead:

- свойства/поля:
 - о поле уровень:
 - наименование — level;
 - тип — int;
 - модификатор доступа — private;
- функционал:
 - о метод TeamLead — конструктор с параметрами;
 - о метод showLevel — метод для вывода закрытого свойства level;
 - о метод showLeadName — метод для вывода имени сотрудника;
 - о метод setName — метод для изменения имени сотрудника;
 - о метод showRole — метод для вывода роли.

Таблица 1 – Иерархия наследования классов

№	Имя класса	Классы-наследники	Модификатор доступа при наследовании	Описание	Номер
1	Employee			базовый класс "Работник"	
		Developer	virtual public		2
		Manager	virtual public		3
2	Developer			производный класс "Разработчик"	
		TeamLead	public		4
3	Manager			производный класс "Менеджер"	
		TeamLead	public		4
4	TeamLead			производный класс "Тимлид"	

3 ОПИСАНИЕ АЛГОРИТМОВ

Согласно этапам разработки, после определения необходимого инструментария в разделе «Метод», составляются подробные описания алгоритмов для методов классов и функций.

3.1 Алгоритм конструктора класса Employee

Функционал: конструктор с параметром.

Параметры: string name - имя.

Алгоритм конструктора представлен в таблице 2.

Таблица 2 – Алгоритм конструктора класса Employee

№	Предикат	Действия	№ перехода
1		присваивает защищённому свойству name параметр name	Ø

3.2 Алгоритм метода getName класса Employee

Функционал: метод для получения значения защищённого свойства.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 3.

Таблица 3 – Алгоритм метода getName класса Employee

№	Предикат	Действия	№ перехода
1		выводит на экран "Employee::showName -> <имя>"	Ø

3.3 Алгоритм метода showRole класса Employee

Функционал: виртуальный метод для вывода роли.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 4.

Таблица 4 – Алгоритм метода *showRole* класса *Employee*

№	Предикат	Действия	№ перехода
1		вывод на экран "Employee::showRole -> Employee"	Ø

3.4 Алгоритм конструктора класса *Developer*

Функционал: конструктор с параметрами.

Параметры: string name - имя; int n - количество задач.

Алгоритм конструктора представлен в таблице 5.

Таблица 5 – Алгоритм конструктора класса *Developer*

№	Предикат	Действия	№ перехода
1		присваивает закрытому свойству ntasks параметр n	Ø

3.5 Алгоритм метода *showTasks* класса *Developer*

Функционал: метод для вывода закрытого свойства.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 6.

Таблица 6 – Алгоритм метода *showTasks* класса *Developer*

№	Предикат	Действия	№ перехода
1		вывод на экран "Developer::showTasks -> <значение>"	Ø

3.6 Алгоритм метода showRole класса Developer

Функционал: метод для вывода на экран роли сотрудника.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 7.

Таблица 7 – Алгоритм метода showRole класса Developer

№	Предикат	Действия	№ перехода
1		вывод на экран "Developer::showRole -> Developer"	Ø

3.7 Алгоритм конструктора класса Manager

Функционал: конструктор с параметрами.

Параметры: string name - имя; int n - количество проектов.

Алгоритм конструктора представлен в таблице 8.

Таблица 8 – Алгоритм конструктора класса Manager

№	Предикат	Действия	№ перехода
1		присваивает закрытому свойству projects параметр n	Ø

3.8 Алгоритм метода showProjects класса Manager

Функционал: метод для вывода закрытого свойства.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 9.

Таблица 9 – Алгоритм метода *showProjects* класса *Manager*

№	Предикат	Действия	№ перехода
1		вывод на экран "Manager::showProjects -> <значение>"	Ø

3.9 Алгоритм метода *showRole* класса *Manager*

Функционал: метод для вывода роли.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 10.

Таблица 10 – Алгоритм метода *showRole* класса *Manager*

№	Предикат	Действия	№ перехода
1		вывод на экран "Manager::showRole -> Manager"	Ø

3.10 Алгоритм конструктора класса *TeamLead*

Функционал: конструктор с параметрами.

Параметры: string name - имя; int ntasks - количество задач; int nprojects - количество проектов; int level - уровень руководства .

Алгоритм конструктора представлен в таблице 11.

Таблица 11 – Алгоритм конструктора класса *TeamLead*

№	Предикат	Действия	№ перехода
1		присваивает закрытому свойству level параметр level	Ø

3.11 Алгоритм метода *showLevel* класса *TeamLead*

Функционал: метод для вывода закрытого свойства level.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 12.

Таблица 12 – Алгоритм метода showLevel класса TeamLead

№	Предикат	Действия	№ перехода
1		вывод на экран "TeamLead::showLevel -> <значение>"	Ø

3.12 Алгоритм метода showLeadName класса TeamLead

Функционал: метод для вывода имени сотрудника.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 13.

Таблица 13 – Алгоритм метода showLeadName класса TeamLead

№	Предикат	Действия	№ перехода
1		вывод на экран "TeamLead::showLeadName -> <имя>"	Ø

3.13 Алгоритм метода setName класса TeamLead

Функционал: метод для изменения имени сотрудника.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 14.

Таблица 14 – Алгоритм метода setName класса TeamLead

№	Предикат	Действия	№ перехода
1		присваивает закрытому свойству name параметр new_name	Ø

3.14 Алгоритм метода showRole класса TeamLead

Функционал: метод для вывода роли.

Параметры: нет.

Возвращаемое значение: void - ничего не возвращает.

Алгоритм метода представлен в таблице 15.

Таблица 15 – Алгоритм метода showRole класса TeamLead

№	Предикат	Действия	№ перехода
1		вывод на экран "TeamLead::showRole -> TeamLead"	Ø

3.15 Алгоритм функции main

Функционал: основная функция программы.

Параметры: нет.

Возвращаемое значение: int - индикатор корректности завершения программы.

Алгоритм функции представлен в таблице 16.

Таблица 16 – Алгоритм функции main

№	Предикат	Действия	№ перехода
1		инициализация переменных name, new_name типа string	2
2		инициализация переменных ntasks, nprojects, level типа int	3
3		ввод с клавиатуры переменных name, ntasks, nprojects, level	4
4		ввод с клавиатуры переменной new_name	5
5		создание объекта tl класса TeamLead с параметрами: name, ntasks, nprojects, level	6
6		вызов метода getName базового класса Employee	7
7		вызов метода showLeadName производного класса TeamLead	8
8		вызов метода setName с параметром new_name	9

№	Предикат	Действия	№ перехода
9		вызов метода getName базового класса Employee	10
10		вызов метода showLeadName производного класса TeamLead	11
11		вызов метода showTasks производного класса Developer	12
12		вызов метода showProjects производного класса Manager	13
13		вызов метода showLevel производного класса TeamLead	14
14		создание указателя на объект dev класса Developer	15
15		создание указателя на объект man класса Manager	16
16		создание указателя на объект emp класса Employee	17
17		присвоение указателю dev адреса объекта tl	18
18		присвоение указателю man адреса объекта tl	19
19		присвоение указателю emp адреса объекта tl	20
20		вызов метода showRole через указатель dev	21
21		вызов метода showRole через указатель man	22
22		вызов метода showRole через указатель emp	∅

4 БЛОК-СХЕМЫ АЛГОРИТМОВ

Представим описание алгоритмов в графическом виде на рисунках 1-5.

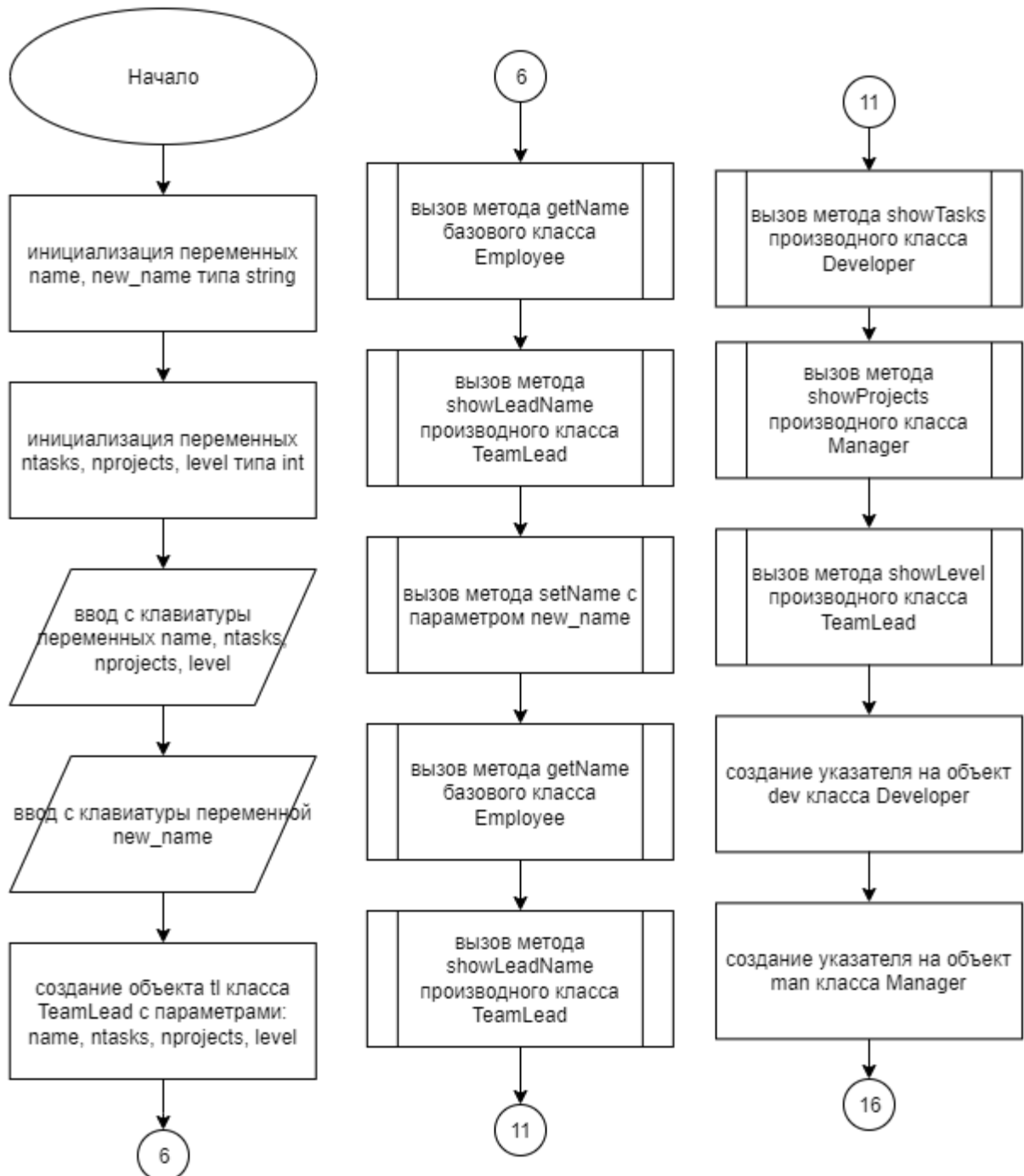


Рисунок 1 – Блок-схема алгоритма

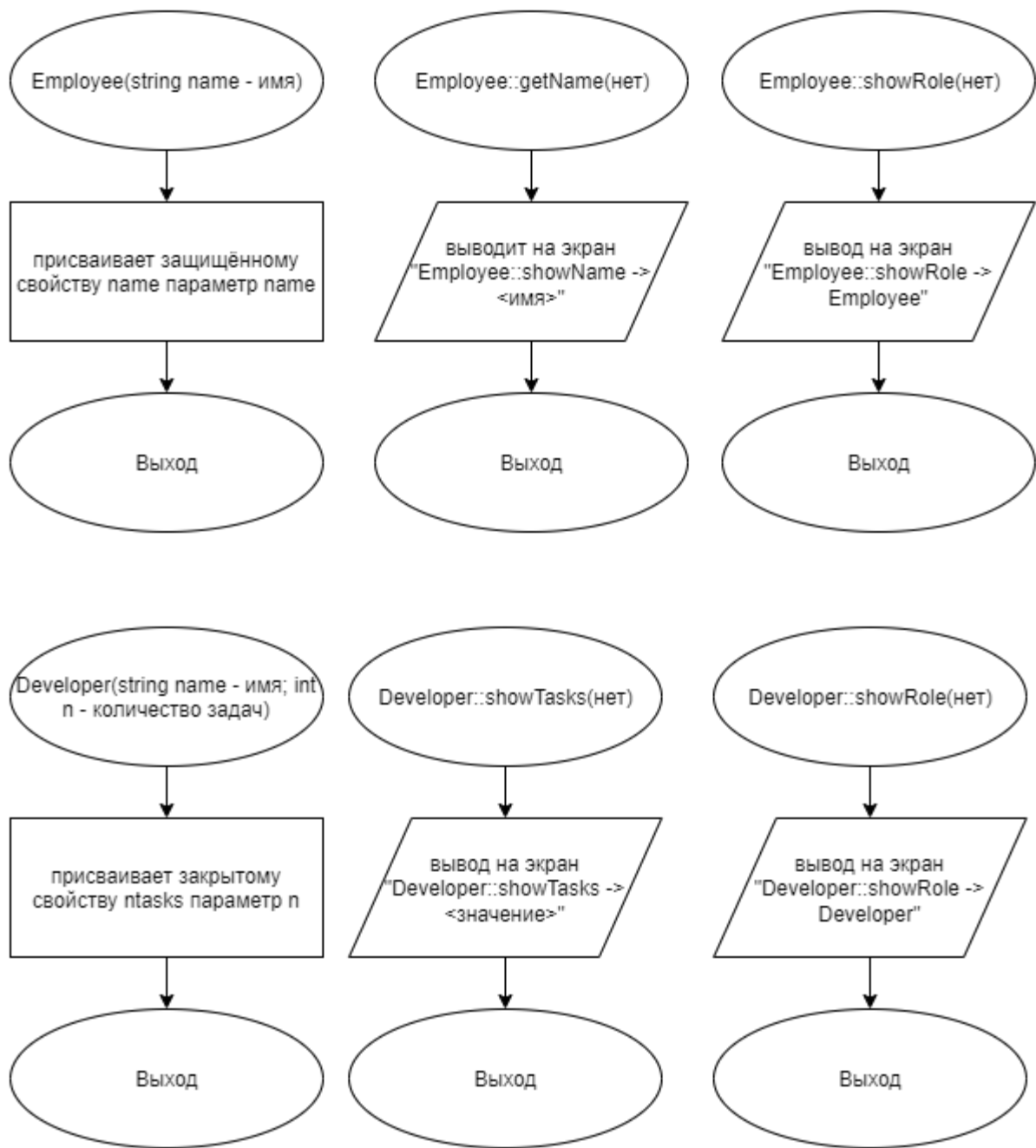


Рисунок 2 – Блок-схема алгоритма

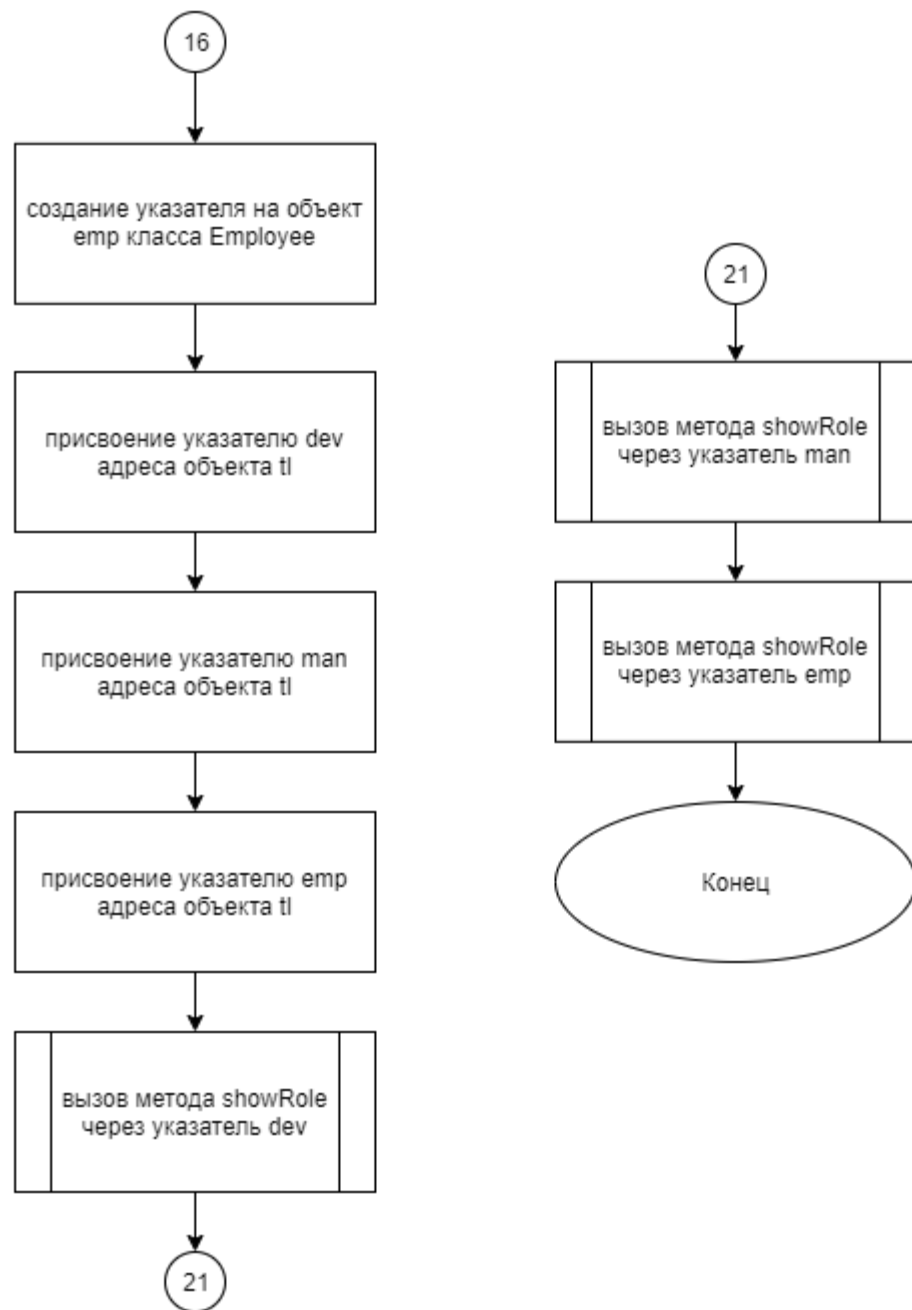


Рисунок 3 – Блок-схема алгоритма

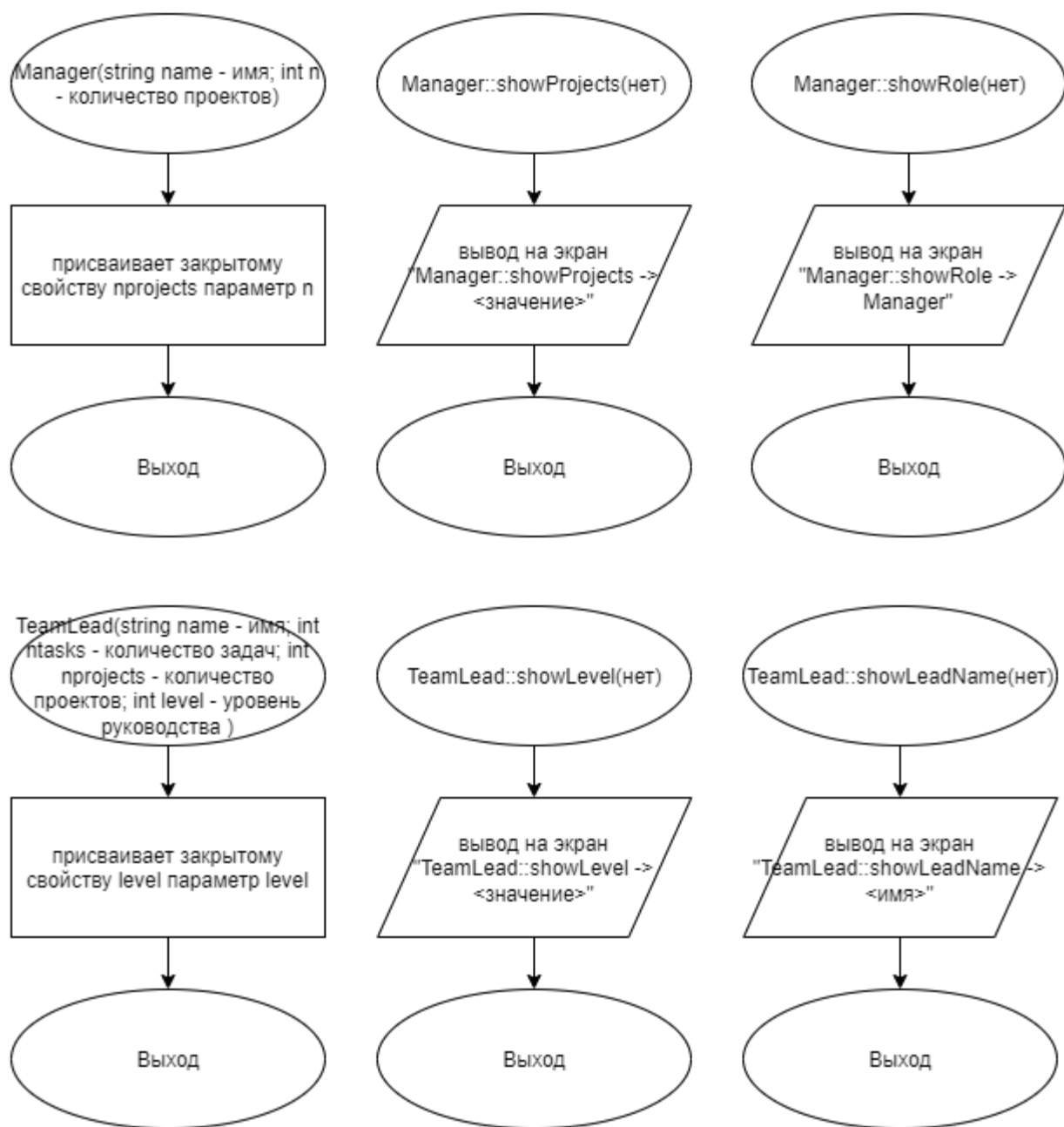


Рисунок 4 – Блок-схема алгоритма

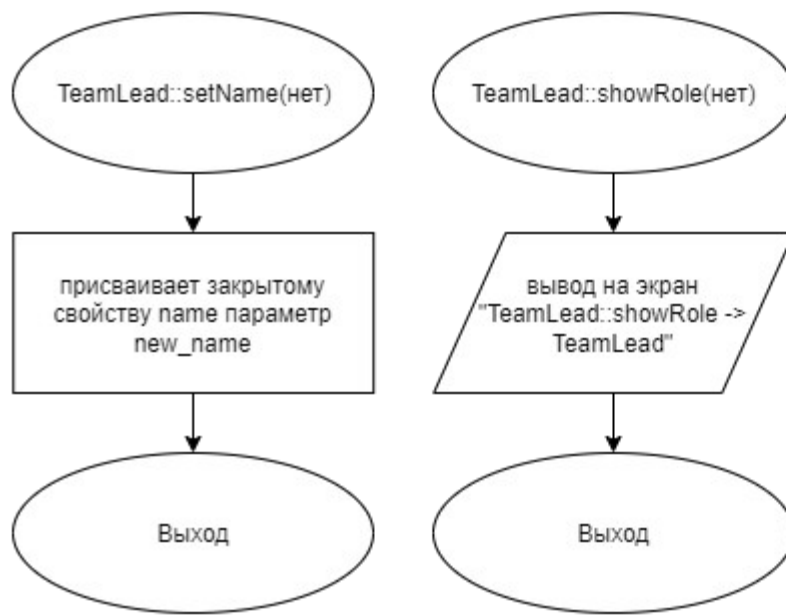


Рисунок 5 – Блок-схема алгоритма

5 КОД ПРОГРАММЫ

Программная реализация алгоритмов для решения задачи представлена ниже.

5.1 Файл Developer.cpp

Листинг 1 – Developer.cpp

```
#include "Developer.h"
#include <string>
using namespace std;

Developer::Developer(string name,int n): Employee(name)
{
    ntasks = n;
}

void Developer::showTasks()
{
    printf("Developer::showTasks -> %d\n", ntasks);
}

void Developer::showRole()
{
    printf("Developer::showRole -> Developer\n");
}
```

5.2 Файл Developer.h

Листинг 2 – Developer.h

```
#ifndef __DEVELOPER__H
#define __DEVELOPER__H

#include "Employee.h"
#include <string>
using namespace std;

class Developer: virtual public Employee
{
private:
    int ntasks;
}
```

```
public:
    Developer(string name, int n);
    void showTasks();
    void showRole();
};

#endif
```

5.3 Файл Employee.cpp

Листинг 3 – Employee.cpp

```
#include "Employee.h"
#include <string>
using namespace std;

Employee::Employee(string name)
{
    this->name = name;
}

void Employee::getName()
{
    printf("Employee::showName -> %s\n", name.c_str());
}

void Employee::showRole()
{
    printf("Employee::showRole -> Employee\n");
}
```

5.4 Файл Employee.h

Листинг 4 – Employee.h

```
#ifndef __EMPLOYEE__H
#define __EMPLOYEE__H

#include <string>
using namespace std;

class Employee
{
protected:
```

```

        string name;

public:
    Employee(string name);
    void getName();
    virtual void showRole();
};

#endif

```

5.5 Файл main.cpp

Листинг 5 – main.cpp

```

#include <stdlib.h>
#include <stdio.h>
#include "Employee.h"
#include "Developer.h"
#include "Manager.h"
#include "TeamLead.h"
#include <string>
#include <iostream>
using namespace std;

int main()
{
    string name, new_name;
    int ntasks, nprojects, level;

    cin >> name >> ntasks >> nprojects >> level;
    cin >> new_name;

    TeamLead tl(name, ntasks, nprojects, level);

    tl.getName();
    tl.showLeadName();

    tl.setName(new_name);

    tl.getName();
    tl.showLeadName();

    tl.showTasks();
    tl.showProjects();
    tl.showLevel();

    Developer *dev;
    Manager *man;
    Employee *emp;
}

```

```

        dev = &tl;
        man = &tl;
        emp = &tl;

        dev->showRole();
        man->showRole();
        emp->showRole();

        return(0);
    }

```

5.6 Файл Manager.cpp

Листинг 6 – Manager.cpp

```

#include "Manager.h"
#include "Employee.h"
#include <string>
using namespace std;

Manager::Manager(string name, int n): Employee(name)
{
    nprojects = n;
}

void Manager::showProjects()
{
    printf("Manager::showProjects -> %d\n", nprojects);
}

void Manager::showRole()
{
    printf("Manager::showRole -> Manager\n");
}

```

5.7 Файл Manager.h

Листинг 7 – Manager.h

```

#ifndef __MANAGER__H
#define __MANAGER__H

#include "Employee.h"
#include <string>
using namespace std;

```

```

class Manager: virtual public Employee
{
private:
    int nprojects;

public:
    Manager(string name, int n);
    void showProjects();
    void showRole();
};

#endif

```

5.8 Файл TeamLead.cpp

Листинг 8 – TeamLead.cpp

```

#include "TeamLead.h"
#include "Developer.h"
#include "Manager.h"
#include "Employee.h"
using namespace std;

TeamLead::TeamLead(string name, int ntasks, int nprojects, int level):
Developer(name, ntasks), Manager(name, nprojects), Employee(name)
{
    this->level = level;
}

void TeamLead::showLevel()
{
    printf("TeamLead::showLevel -> %d\n", level);
}

void TeamLead::showLeadName()
{
    printf("TeamLead::showLeadName -> %s\n", name.c_str());
}

void TeamLead::setName(string new_name)
{
    name = new_name;
}

void TeamLead::showRole()
{
    printf("TeamLead::showRole -> TeamLead\n");
}

```

5.9 Файл TeamLead.h

Листинг 9 – TeamLead.h

```
#ifndef __TEAMLEAD__H
#define __TEAMLEAD__H

#include "Developer.h"
#include "Manager.h"
#include <string>
using namespace std;

class TeamLead: public Developer, public Manager
{
private:
    int level;

public:
    TeamLead(string name, int ntasks, int nprojects,int level);
    void showLevel();
    void showLeadName();
    void setName(string new_name);
    void showRole();
};

#endif
```

6 ТЕСТИРОВАНИЕ

Результат тестирования программы представлен в таблице 17.

Таблица 17 – Результат тестирования программы

Входные данные	Ожидаемые выходные данные	Фактические выходные данные
Ivan 12 3 2 Petr	Employee::showName - > Ivan TeamLead::showLeadName -> Ivan Employee::showName - > Petr TeamLead::showLeadName -> Petr Developer::showTasks -> 12 Manager::showProjects -> 3 TeamLead::showLevel -> 2 TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead	Employee::showName - > Ivan TeamLead::showLeadName -> Ivan Employee::showName - > Petr TeamLead::showLeadName -> Petr Developer::showTasks -> 12 Manager::showProjects -> 3 TeamLead::showLevel -> 2 TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead TeamLead::showRole - > TeamLead

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. ГОСТ 19 Единая система программной документации.
2. Методическое пособие студента для выполнения практических заданий, контрольных и курсовых работ по дисциплине «Объектно-ориентированное программирование» [Электронный ресурс] – URL: https://mirea.aco-avvora.ru/student/files/methodichescoc_posobie_dlya_laboratornyh_rabot_3.pdf (дата обращения 05.05.2021).
3. Приложение к методическому пособию студента по выполнению заданий в рамках курса «Объектно-ориентированное программирование» [Электронный ресурс]. URL: https://mirea.aco-avvora.ru/student/files/Prilozheniye_k_methodichke.pdf (дата обращения 05.05.2021).
4. Шилдт Г. С++: базовый курс. 3-е изд. Пер. с англ.. — М.: Вильямс, 2019. — 624 с.
5. Видео лекции по курсу «Объектно-ориентированное программирование» [Электронный ресурс]. АСО «Аврора».
6. Антик М.И. Дискретная математика [Электронный ресурс]: Учебное пособие /Антик М.И., Казанцева Л.В. — М.: МИРЭА — Российский технологический университет, 2018 — 1 электрон. опт. диск (CD-ROM).